0239. Sliding Window Maximum

Category: Sliding Window | LeetCode: https://leetcode.com/problems/sliding-wi

Problem

- Return the maximum of every window of size k as it slides across nums.
- Example: [1,3,-1,-3,5,3,6,7], k = 3 → [3,3,5,5,6,7].

Intuition

- Keep a deque of indices whose values are decreasing from front to back.
- A new value evicts smaller values from the back (they can never be a max again while it exists).
- The front is the current max; drop it when it leaves the window.

Brute Force

- Scan each window: O(nk). A max-heap with lazy deletion gives O(n log n).

Optimal Approach

- For each i: pop back while nums[back] <= nums[i]; push i.
- If front <= i - k, pop front.
- If i >= k-1, output nums[front].

Worked Example

```
nums = [1,3,-1,-3,5,3,6,7], k = 3 (deque shows values).

i=0 1     dq=[1]
i=1 3     dq=[3]
i=2 -1    dq=[3,-1]       out 3
i=3 -3    dq=[3,-1,-3]    out 3
i=4 5     dq=[5]          out 5
i=5 3     dq=[5,3]        out 5
i=6 6     dq=[6]          out 6
i=7 7     dq=[7]          out 7
```

Java Solution

```java
public class SlidingWindowMaximum {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int[] out = new int[nums.length - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        for (int i = 0; i < nums.length; i++) {
            while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast();
            dq.offerLast(i);
            if (dq.peekFirst() <= i - k) dq.pollFirst();
            if (i >= k - 1) out[i - k + 1] = nums[dq.peekFirst()];
        }
        return out;
    }
}
```

Complexity

- Time: O(n), each index is pushed and popped once. Space: O(k).

Pitfalls

- Store indices, not values, so you can tell when the front expires.